

ServiceNow MCP Gateway (Local)

Full Setup & Usage Guide

Built by @AlbyTusharM

A local app that turns any ServiceNow REST API into MCP tools, allowing Claude Desktop, Claude Code, or any MCP client to call your instance directly.

Repository: github.com/yvain13/servicenow-mcp-gateway

Step 1: Install the Prerequisites

You need two things: **Git** (to pull the repo) and **Node.js** (to run it). The gateway is built on Node 22 types, so use Node 18 or newer, ideally Node 20+.

Git

- **macOS:** `git --version` will prompt you to install Xcode command line tools if Git is missing. Or `brew install git`.
- **Windows:** Download from git-scm.com/download/win and run the installer.
- **Linux:** `sudo apt install git` (Debian/Ubuntu) or your distro's package manager.

```
git --version
```

Node.js

Grab the LTS build from nodejs.org. The installer also gives you `npm`. If you switch Node versions a lot, use a version manager instead (like `nvm` for macOS/Linux or `nvm-windows`).

```
node --version    # v20.x or higher
npm --version
```

Step 2: Get the Repo Running

Clone and install

```
git clone https://github.com/yvain13/ServiceNow-mcp-gateway.git
cd ServiceNow-mcp-gateway
npm install
```

`npm install` pulls in five runtime dependencies: the official MCP TypeScript SDK, Express, CORS, dotenv, and Zod.

Create your env file

```
cp .env.example .env
```

Open `.env` and put in your real ServiceNow details. A free Personal Developer Instance from developer.servicenow.com is perfect for following along.

```
MANAGEMENT_PORT=9009
SN_INSTANCE_URL=https://devXXXXXX.service-now.com
SN_USERNAME=your_service_account
SN_PASSWORD=your_password
```

Run it

```
npm run dev
```

`npm run dev` runs under `tsx watch`, so it hot-reloads whenever you change a source file. For a production-style run instead, use `npm run build && npm start`.

Management UI: Open `http://localhost:9009` in your browser. This is the control panel. On startup, the gateway also auto-starts every MCP server already saved in `data/store.json`.

Step 3: Understanding the Gateway Architecture

There are three layers to keep straight:

- 1. **The management process:** One Express app serving the web UI plus a REST API under `/api`, on `MANAGEMENT_PORT`. This is the control panel.
- 2. **The MCP servers:** Each MCP server you create is its own Express app on its own port, exposing `/mcp` through `StreamableHTTPServerTransport`.
- 3. **The store:** Servers and tools are persisted to `data/store.json`. The handlers read this file live, so editing a tool in the UI takes effect immediately without a restart.

Environment Variables

Variable	Required	What it does
----------	----------	--------------

<code>MANAGEMENT_PORT</code>	No (defaults 3001)	Port for the management UI and REST API
<code>SN_INSTANCE_URL</code>	Yes	Your instance, e.g. <code>https://devXXXXXX.service-now.com</code>
<code>SN_USERNAME</code>	Yes	The user the gateway authenticates as
<code>SN_PASSWORD</code>	Yes	Password for that user

Managing Servers and Tools

Adding an MCP server: In the UI, click "Create MCP Server", give it a name (e.g. `incident-mcp`) and a port (e.g. `7801`). Use Start/Stop to control it.

Adding a tool: Select a server, click "Add Tool", and configure the Tool name, Description, API type, Method, Endpoint, and Parameters. Click "Save Tool" to apply instantly.

Step 4: Configuring Tool Parameters

Each parameter has a Name, Description, Type, Required flag, and a "Send as" location.

The four "Send as" options:

- **Auto:** Sent in the body for POST/PATCH, or query string for GET/DELETE.
- **Body:** Forces value into the JSON request body (e.g., `short_description`).
- **Query:** Forces value into the URL query string (e.g., filters).
- **Path:** Substitutes value into a `{placeholder}` in the endpoint (e.g., `/api/now/table/incident/{sys_id}`).

Safety Behaviors:

1. *Path auto-detection:* Any `{name}` in the endpoint is treated as a path slot automatically.
2. *Mismatch guard:* Marking a parameter as Path without a matching `{name}` throws a clear error.

Table API Convenience Mapping

When the API type is "Table API", the gateway maps clean parameter names to `sysparm_*` automatically:

You write	Gateway sends
<code>query</code>	<code>sysparm_query</code>
<code>fields</code>	<code>sysparm_fields</code>
<code>limit</code>	<code>sysparm_limit</code>
<code>order_by</code>	<code>sysparm_orderby</code>
<code>display_value</code>	<code>sysparm_display_value</code>

Step 5: The 6 Default Servers

The gateway ships with six pre-seeded servers in `data/store.json` . All use the Table API.

1. incident-mcp (Port 7801)

Tool	Method	Endpoint
<code>get_incident</code>	GET	<code>/api/now/table/incident</code>
<code>create_incident</code>	POST	<code>/api/now/table/incident</code>
<code>update_incident</code>	PATCH	<code>/api/now/table/incident/{sys_id}</code>

2. change-ops (Port 7820)

Read, create, and update change requests. `create_change` includes implementation plans, risk analysis, and backout plans.

3. cmdb_server (Port 7850)

Pull configuration item data from the CMDB via `get_cmdb_ci` .

Developer Trio

- **business_rule_mcp (Port 8110):** Full CRUD over Business Rules (`sys_script`).
- **script_include_mcp (Port 8120):** Full CRUD over Script Includes (`sys_script_include`).
- **client_script_mcp (Port 8140):** Full CRUD over Client Scripts (`sys_script_client`).

Step 6: Connect to Claude Desktop

The gateway speaks Streamable HTTP. Claude Desktop speaks stdio. You bridge them with `mcp-remote` , generated automatically via the UI.

Get the snippet

In the UI, select a server and click **Config Snippet**, then Copy.

```
{
  "mcpServers": {
    "incident-mcp": {
      "command": "npx",
      "args": ["mcp-remote", "http://localhost:7801/mcp"]
    }
  }
}
```

Add it to Claude Desktop

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

Paste the snippet into the file. Save, fully quit, and reopen Claude Desktop. Keep the gateway running (`npm run dev`) so Claude can reach the local ports.

Step 7: A Worked Example

With `incident-mcp` wired into Claude Desktop, try this prompt:

"Get incident INC0010001 and tell me its priority and short description."

Claude calls `get_incident` with `number = INC0010001`. The gateway fetches the record from your instance, and Claude summarizes it.

Next, try writing data:

"Create a P3 incident: short description 'Login page intermittently 500s', description 'Users report sporadic 500 errors on the SSO login page during peak hours.'"

Step 8: Connecting Other Clients

Claude Code

Claude Code can talk HTTP transport directly, no `mcp-remote` bridge needed:

```
claude mcp add --transport http incident-mcp http://localhost:7801/mcp
```

Inside a session, type `/mcp` to list connected servers. Try `get incident INC0010001`.

GitHub Copilot (VS Code)

VS Code supports MCP servers via a workspace `.vscode/mcp.json` file:

```
{
  "servers": {
    "incident-mcp": {
      "type": "http",
      "url": "http://localhost:7801/mcp"
    }
  }
}
```

Reload the window and open Copilot Chat in Agent mode.

Step 9: Connecting Atlassian MCP (for Automations)

To set up Jira for the `snow-deliver` automation, add Atlassian's hosted Remote MCP Server to Claude Code over SSE:

```
claude mcp add --transport sse atlassian https://mcp.atlassian.com/v1/sse
```

The first call triggers an OAuth browser flow. After approval, test it:

```
"Fetch Jira issue SCRUM-1 and summarize the requirement."
```


Step 10: The Big Picture Automations

Combine the gateway's ServiceNow tools with Claude Code skills for fully agentic workflows.

Skill 1: remediate-incident (Ops-focused)

Trigger: `remediate incident INC0000055`

Workflow: Investigates the incident, pulls the affected CI, generates a complete change request, and writes a work note back to the incident without human intervention. Uses `incident-mcp`, `cmdb_server`, and `change-ops`.

Skill 2: snow-deliver (Dev-focused)

Trigger: `deliver SCRUM-1`

Workflow: Reads a Jira ticket, checks if the artifact already exists in ServiceNow. If not, it builds the new artifact (Business Rule, Script Include, etc.), and closes the Jira ticket automatically. Uses the Atlassian MCP plus the developer trio servers.

Quick Reference

Server	Port	MCP Endpoint
<code>incident-mcp</code>	7801	<code>http://localhost:7801/mcp</code>
<code>change-ops</code>	7820	<code>http://localhost:7820/mcp</code>
<code>cmdb_server</code>	7850	<code>http://localhost:7850/mcp</code>
<code>business_rule_mcp</code>	8110	<code>http://localhost:8110/mcp</code>
<code>script_include_mcp</code>	8120	<code>http://localhost:8120/mcp</code>
<code>client_script_mcp</code>	8140	<code>http://localhost:8140/mcp</code>

Command	What it does
---------	--------------

```
npm install
```

Install dependencies

```
npm run dev
```

Run with hot reload (tsx watch)

```
npm run build
```

Compile TypeScript to `dist/`

```
npm start
```

Run the compiled output